

ANSH THAPA

B.TECH CSE

CU24250224

Experiment No. 5

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, and computer vision applications.

Purpose

Feature extraction is a fundamental step in computer vision that transforms raw image data into meaningful descriptors suitable for object recognition, image matching, tracking, and classification. Robust feature descriptors should be invariant to changes in scale, rotation, illumination, and viewpoint.

In this experiment, students will implement SIFT and HOG feature extraction techniques on real-world images and compare their performance in terms of feature detection, robustness, computational complexity, and suitability for different computer vision applications.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), scikit-image, NumPy, and Matplotlib
- Understanding of image gradients, keypoints, and feature descriptors
- Jupyter Notebook, Google Colab, or Visual Studio Code

```
import cv2
import numpy as np
```

```
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
from google.colab import files

uploaded = files.upload()
filename = list(uploaded.keys())[0]

img_color = cv2.imread(filename)
img_color = cv2.cvtColor(img_color, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6,6))
plt.imshow(img_color)
plt.title("Original Image")
plt.axis('off')
plt.show()

print("Image shape:", img_color.shape)
```

[Choose Files](#)

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 2025_Nitro_Default_3840x2400.jpg to 2025_Nitro_Default_3840x2400 (4).j

Original Image



```
gray = cv2.cvtColor(img_color, cv2.COLOR_RGB2GRAY)
gray_eq = cv2.equalizeHist(gray)
gray_eq = cv2.GaussianBlur(gray_eq, (3,3), 0)

fig, ax = plt.subplots(1, 3, figsize=(15,5))
ax[0].imshow(img_color); ax[0].set_title("Original (Color)"); ax[0].axis('off')
ax[1].imshow(gray, cmap='gray'); ax[1].set_title("Grayscale"); ax[1].axis('off')
ax[2].imshow(gray_eq, cmap='gray'); ax[2].set_title("After Preprocessing");
plt.show()
```

Original (Color)



Grayscale



After Preprocessing



```
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray_eq, None)

print("Total keypoints detected:", len(keypoints))
print("Descriptor matrix shape:", descriptors.shape)
```

```
Total keypoints detected: 33191
Descriptor matrix shape: (33191, 128)
```

```
img_kp = cv2.drawKeypoints(
    img_color, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

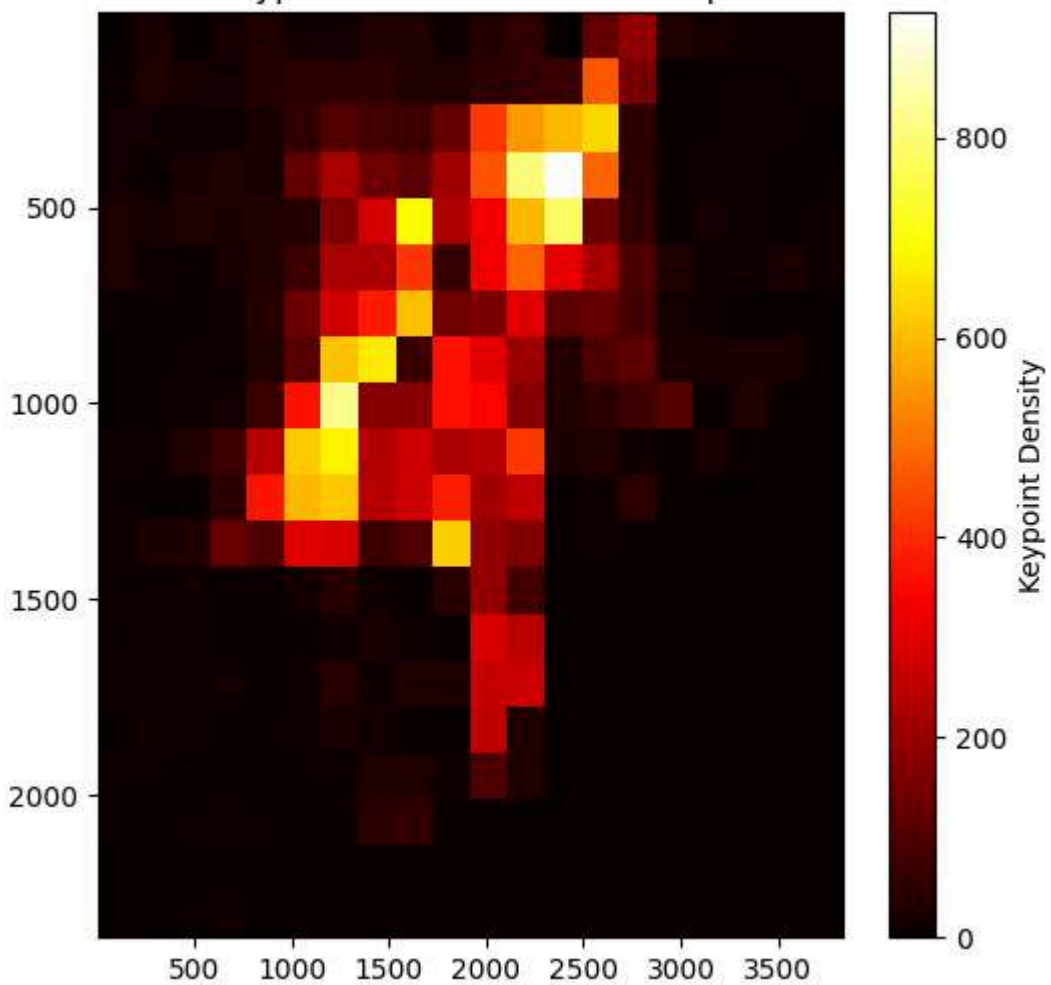
plt.figure(figsize=(8,8))
plt.imshow(img_kp)
plt.title(f"SIFT Keypoints (Total: {len(keypoints)})")
plt.axis('off')
plt.show()

kp_coords = np.array([kp.pt for kp in keypoints])
plt.figure(figsize=(6,6))
plt.hist2d(kp_coords[:,0], kp_coords[:,1], bins=20, cmap='hot')
plt.gca().invert_yaxis()
plt.colorbar(label='Keypoint Density')
plt.title("Keypoint Distribution Heatmap")
plt.show()
```

SIFT Keypoints (Total: 33191)



Keypoint Distribution Heatmap



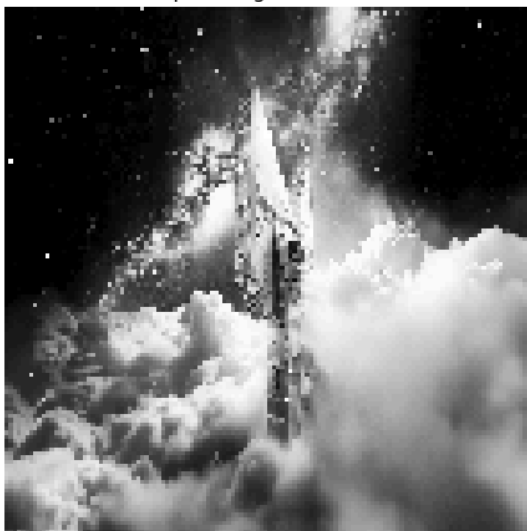
```
gray_resized = cv2.resize(gray_eq, (128, 128))
```

```
hog_features, hog_image = hog(  
    gray_resized,  
    orientations=9,  
    pixels_per_cell=(8, 8),  
    cells_per_block=(2, 2),  
    block_norm='L2-Hys',  
    visualize=True,  
    feature_vector=True  
)  
  
print("HOG feature vector length:", len(hog_features))  
print("HOG image shape:", hog_image.shape)
```

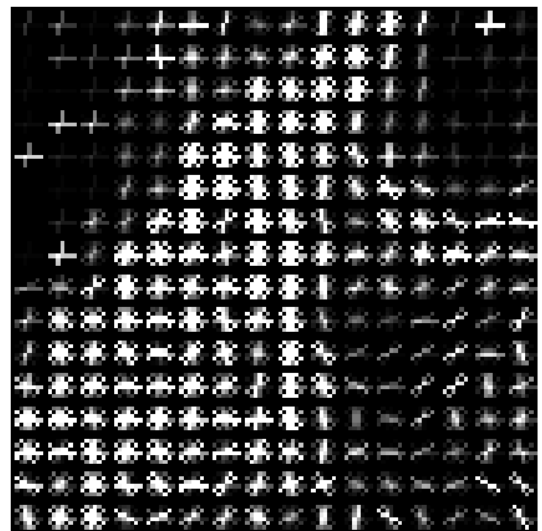
```
HOG feature vector length: 8100  
HOG image shape: (128, 128)
```

```
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))  
  
fig, ax = plt.subplots(1, 2, figsize=(12,6))  
ax[0].imshow(gray_resized, cmap='gray')  
ax[0].set_title("Input Image (Resized)")  
ax[0].axis('off')  
  
ax[1].imshow(hog_image_rescaled, cmap='gray')  
ax[1].set_title("HOG Feature Visualization")  
ax[1].axis('off')  
plt.show()
```

Input Image (Resized)



HOG Feature Visualization



```

print("="*70)
print(f"{'Property':<25}{ 'SIFT':<25}{ 'HOG' }")
print("="*70)
print(f"{'Scale Invariance':<25}{ 'Yes':<25}{ 'No' }")
print(f"{'Rotation Invariance':<25}{ 'Yes':<25}{ 'No' }")
print(f"{'Output Type':<25}{ 'Sparse keypoints':<25}{ 'Dense feature vector' }")
print(f"{'Feature Length':<25}{ '128 per keypoint':<25}{len(hog_features)}")
print(f"{'Speed':<25}{ 'Slower':<25}{ 'Faster' }")
print(f"{'Best Use':<25}{ 'Matching/Recognition':<25}{ 'Detection (shape-based' }")
print("="*70)

```

```

=====
Property                SIFT                HOG
=====
Scale Invariance        Yes                 No
Rotation Invariance     Yes                 No
Output Type             Sparse keypoints    Dense feature vector
Feature Length          128 per keypoint   8100 (whole image)
Speed                   Slower             Faster
Best Use                Matching/Recognition Detection (shape-based)
=====

```

```

uploaded2 = files.upload()
filename2 = list(uploaded2.keys())[0]

img2_color = cv2.imread(filename2)
img2_color = cv2.cvtColor(img2_color, cv2.COLOR_BGR2RGB)
gray2 = cv2.cvtColor(img2_color, cv2.COLOR_RGB2GRAY)

kp2, des2 = sift.detectAndCompute(gray2, None)

bf = cv2.BFMatcher()
matches = bf.knnMatch(descriptors, des2, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good matches found:", len(good_matches))

result = cv2.drawMatches(
    img_color, keypoints, img2_color, kp2,
    good_matches[:30], None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16,8))
plt.imshow(result)
plt.title("SIFT Feature Matching Between Two Images")
plt.axis('off')
plt.show()

```


[Choose Files](#)

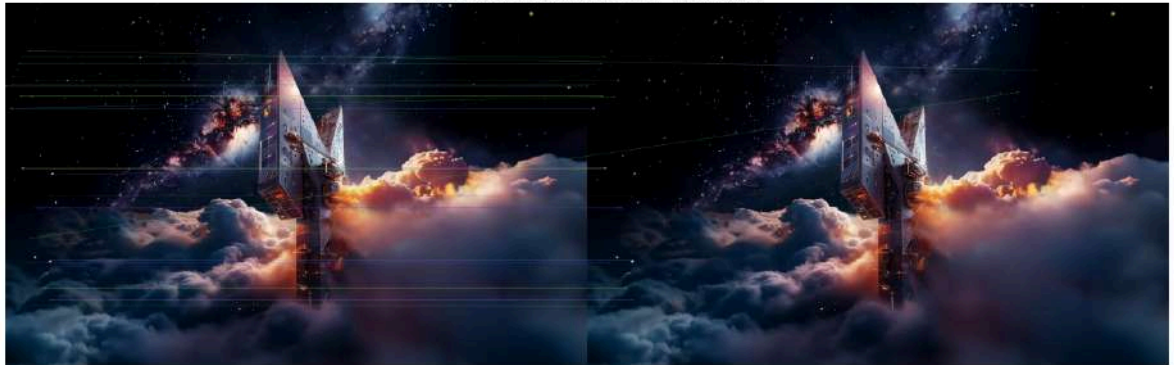
No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 2025_Nitro_Default_3840x2400.jpg to 2025_Nitro_Default_3840x2400 (5).j

Good matches found: 3311

SIFT Feature Matching Between Two Images



SIFT — Strengths and Limitations

Strengths:

- Scale-invariant and rotation-invariant
- Accurate matching across different viewpoints/angles
- High-quality, distinctive 128-dimensional descriptors

Limitations:

- Computationally expensive, especially on large images
- Too heavy for real-time applications
- Fewer keypoints on textureless/flat images

HOG — Strengths and Limitations

Strengths:

- Fast and computationally efficient
- Captures object shape/structure well
- Reliable with classifiers like SVM for detection tasks (e.g. pedestrian detection)

Limitations:

- Not scale-invariant or rotation-invariant

- Performance drops when object orientation changes
- Follows a fixed grid structure, not suited for flexible matching

✓ Final Observations

1. SIFT provides sparse, distinctive keypoints ideal for matching and recognition, especially when image scale/angle differs.
2. HOG provides a dense feature vector for the whole image describing overall shape/structure — better suited for detection tasks.
3. SIFT offers rotation and scale invariance, while HOG is sensitive to these.
4. HOG is computationally lighter, making it more practical for real-time detection systems.

Applications

SIFT use cases:

- Panorama/image stitching
- Object recognition across different scales and viewpoints
- 3D reconstruction, SLAM (robot navigation)
- Augmented reality (marker detection)

HOG use cases:

- Pedestrian detection (classic HOG + SVM combination)
- Vehicle detection
- Real-time object detection systems

Conclusion

SIFT is best when precise matching is needed across different scales/rotations, while HOG is best for fast, shape-based object detection where invariance is less critical.

Questions

1. What is feature extraction, and why is it important in computer vision?

ANS Feature extraction is the process of identifying important information or patterns from an image, such as edges, corners, shapes, textures, and colors.

It is important because it:

- Reduces unnecessary information in an image.
- Makes image processing faster.
- Helps in identifying and comparing objects.

- Improves the performance of classification and object detection systems.

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

ANS SIFT (Scale-Invariant Feature Transform) is a feature extraction technique used to find important points in an image.

Its working mainly involves these steps:

- **Scale-space construction:** The image is processed at different scales to detect features of different sizes.
- **Keypoint detection:** Important points such as corners and distinctive regions are identified.
- **Keypoint localization:** Weak or unstable points are removed.
- **Orientation assignment:** An orientation is assigned to each keypoint based on the surrounding image region.
- **Descriptor creation:** A numerical description of the area around each keypoint is created.
- **Feature matching:** These descriptors can be compared with descriptors from another image to find matching features.

SIFT is useful because the same feature can usually be recognized even if the image is resized or rotated.

3. What are keypoints and feature descriptors in image analysis?

ANS Keypoints are important or distinctive points in an image, such as corners, edges, or unique regions.

- A feature descriptor is a numerical representation that describes the area around a keypoint.

For example:

Keypoint: A corner of a building.

Descriptor: Numerical information describing the pattern around that corner.

Keypoints help locate important regions, while descriptors help compare and match those regions between images.

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

ANS HOG (Histogram of Oriented Gradients) is a feature descriptor that represents an image using the direction and strength of edges.

Working:

1. The image is divided into small cells.

2. Gradients (changes in intensity) are calculated.
3. The direction of the edges is found.
4. A histogram of edge directions is created for each cell.
5. These histograms are combined to form the HOG feature.

Significance:

HOG is very useful for detecting the shape and outline of objects, especially humans. It became popular for human and pedestrian detection.

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

ANS SIFT and HOG are both feature extraction techniques, but they are designed for different purposes.

Robustness: SIFT is highly robust to changes in scale and rotation. It can identify the same feature even when an image is resized or rotated. HOG is mainly designed to capture the shape and edge information of an object. It works well when the object's shape is important, but it is less effective than SIFT when there are large changes in scale or rotation.

Computational Complexity: SIFT is more computationally expensive because it searches for keypoints at different scales and creates detailed descriptors for each keypoint. HOG is comparatively simpler and faster because it calculates image gradients and creates histograms of their orientations.

Practical Applications: SIFT is commonly used for image matching, panorama stitching, object recognition, and image registration. HOG is commonly used for pedestrian detection, human detection, and shape-based object detection.

In short: SIFT is better when images may have different scales or rotations, while HOG is better for shape-based object detection.

6. Why is SIFT considered invariant to scale and rotation?

ANS SIFT is scale-invariant because it detects features at different image sizes using a scale-space representation.

It is rotation-invariant because SIFT assigns an orientation to each keypoint. The descriptor is then created relative to this orientation. Therefore, even if an image is resized or rotated, the same important feature can still be detected and matched.

7. Mention three real-world applications where HOG descriptors are commonly used.

ANS Three common applications of HOG are:

1. Pedestrian detection in surveillance systems.
2. Human detection in autonomous vehicles.

3. Object detection in computer vision systems.

HOG is especially useful when the shape and edges of an object are important.

8. Why is feature extraction performed before image classification or object detection?

ANS Feature extraction is performed before classification or detection because a raw image contains a large amount of information, including unnecessary details.

Feature extraction:

- Selects important information.
- Reduces the size of the data.
- Makes processing faster.
- Helps the classifier understand important patterns.
- Can improve classification and detection accuracy.

For example, instead of processing every pixel of a person's image, a system can use edges, shapes, and other useful features.

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

ANS Handcrafted features such as SIFT and HOG are designed using predefined mathematical methods.

Advantages:

1. Easy to understand.
2. Usually require less training data.
3. Can work well for specific tasks.
4. Require less computational power than large deep learning models.
5. Their behavior is easier to analyze.

Limitations:

1. They may not work well in complex environments.
2. Their performance depends on manually designed rules.
3. They may not capture high-level features effectively.
4. Deep learning methods can usually learn better features automatically from large datasets.

In short: Handcrafted features are simpler and require less data, while deep learning features are usually more powerful but require more data and computational resources.

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

ANS Feature extraction helps these systems by converting images into useful and measurable features.

Image Matching: Features such as SIFT keypoints and descriptors are compared between two images to find matching regions.

Face Recognition: Features related to the shape and structure of a face are extracted and compared with stored face features.

Object Detection: Features such as edges, shapes, and textures help identify and locate objects in an image.